

LISTA DE EXERCÍCIOS – CAPÍTULO 1 CONCEITOS BÁSICOS & INTRODUÇÃO À LINGUAGEM C

Prezado(a) estudante, realize esta lista de exercícios com tranquilidade e atenção aos detalhes, elabore um repositório no GitHub para responder essa lista de exercício e as demais listas da disciplina. Nesse repositório crie uma pasta para cada lista. Para as respostas das questões escritas objetivas e abertas crie um documento de texto com as respostas, já para as questões de código elabore um arquivo .c para cada questão de código. O objetivo desta atividade é consolidar os conhecimentos fundamentais da Linguagem C estudados no Capítulo 1 do livro 'Treinamento em Linguagem C' e apresentados em sala de aula. Esta lista foca na estrutura básica de programas, funções de saída padrão (printf), constantes, variáveis, modificadores de tipo e a configuração do ambiente de desenvolvimento. Bom trabalho!.

Questão 01. Escreva um programa completo em C que declare uma variável do tipo inteiro, atribua um valor a ela (como o seu ano de nascimento ou o ano letivo corrente) e a imprima na tela junto com uma mensagem de texto explicativa utilizando a função printf() com o especificador de formato correspondente.

Questão 02. Faça um programa em C que declare uma variável de ponto flutuante de precisão simples (float), atribua a ela um valor constante real de sua preferência (como o valor do número de Euler 'e' = 2.71828) e exiba o resultado no console formatado com exatamente três casas decimais de precisão.

Questão 03. O uso correto de comentários é fundamental para documentar e tornar o código compreensível. Com base nos tipos de comentários estudados (múltiplas linhas e linha única), escreva um programa simples em C e documente-o de forma clara. Siga o modelo de formatação de código ilustrado abaixo:

```
/* Esse programa mostra o uso de comentários em várias linhas
 * e mostra também o uso de comentários em uma única linha
 *
 *           Primeiro programa
 * *****/

/* Prog1.C */
```

```
#include <stdio.h> /* Para printf() */
#include <stdlib.h> /* Para system() */

int main()      /* Função main */
{ /* início do corpo da função main */
    printf("Primeiro programa."); /* Chamada à função printf */
    system("PAUSE");             /* Chamada à função system */
    return 0;
} /* Fim do corpo da função main */
```

Questão 04. Um estudante iniciante de programação em C escreveu o programa abaixo e encontrou diversos erros que impedem a sua compilação. Analise o código atentamente, aponte cada um dos erros presentes e escreva a versão corrigida e funcional desse programa:

```
#include <stdio.h>
#include <stdlib.h>;
int Main{}
(
    printf( Existem %d semanas no ano.,52);
    cout << endl;
    system("PAUSE");
    return 0;
)
```

Questão 05. Analise o seguinte trecho de código em C. Sob a perspectiva do padrão ANSI C, o programa está correto para compilação e execução imediata? Caso negativo, descreva quais elementos cruciais e diretivas estão faltando no código abaixo:

```
main()
{
    printf("Linguagem C");
    system("pause");
}
```

Questão 06. Identifique e liste todos os erros de sintaxe (que violam as regras da linguagem C) e de lógica contidos no programa abaixo:

```
main()
{
    int a=1; b=2; c=3:
    printf("Os números são: %d%d%d\n, a, b, c, d);
    system("pause");
}
```

Questão 07. Descreva a saída exata (incluindo quebras de linha e tabulações) que será impressa no console por cada uma das seguintes instruções independentes do printf():

- a) printf("\n\tBom dia! Shirley.");
- b) printf("Você já tomou café? \n");
- c) printf("\n\nA solução não existe!\nNão insista.");
- d) printf("Duas\tlinhas\tde\tsaída\nou\tuma?");
- e) printf("%s\n%s\n%s\n", "um", "dois", "três");

Questão 08. Explique detalhadamente o comportamento do programa abaixo quando executado no console. Apresente qual será a saída exata gerada pelas sequências de escape utilizadas no formato de controle:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("\n\t\"Primeiro programa\"");
    system("PAUSE");
    return 0;
}
```

Questão 09. Determine a saída exata do programa a seguir e explique como o compilador C interpreta os argumentos do tipo caractere simples ('\n', '\t', '\") passados para o modificador %c:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("%c%c%cPrimeiro programa", '\n', '\t', '\");
    printf("%c", "\");
    system("PAUSE");
    return 0;
}
```

Questão 10. A Linguagem C é conhecida por ser sensível a caixa alta e baixa (case sensitive). Explique o significado prático desse conceito. Identificadores como 'peso', 'Peso' e 'PESO' representam a mesma variável na memória? Assinale a alternativa correta e complemente com sua justificativa:

- a) Depende exclusivamente da implementação do compilador utilizado no sistema.
- b) Verdadeiro (a linguagem C diferencia rigorosamente letras maiúsculas de minúsculas).
- c) Falso (letras maiúsculas e minúsculas são interpretadas como equivalentes pelo compilador).

Questão 11. Para cada um dos valores constantes descritos na tabela abaixo, indique a classificação correta (por exemplo: constante inteira decimal, constante de ponto flutuante, constante de caractere, constante string ou sequência de escape) e o tipo de dado base correspondente em C (como char, int, float, double):

Constante	Classificação (Tipo de Constante)	Tipo Base em C
\r	[Preencher]	[Preencher]
2130	[Preencher]	[Preencher]
-123	[Preencher]	[Preencher]
33.28	[Preencher]	[Preencher]
0XFA	[Preencher]	[Preencher]
0101	[Preencher]	[Preencher]
2.0e30	[Preencher]	[Preencher]
\xDC	[Preencher]	[Preencher]
'\"'	[Preencher]	[Preencher]
'\\'	[Preencher]	[Preencher]
'F'	[Preencher]	[Preencher]
0	[Preencher]	[Preencher]
'\0'	[Preencher]	[Preencher]
"F"	[Preencher]	[Preencher]
-4567.89	[Preencher]	[Preencher]

Questão 12. A declaração de variáveis define o tipo e o identificador de cada espaço reservado na memória. Analise cada uma das declarações na tabela a seguir, preencha o seu status (Correto ou Incorreto) e, caso seja incorreto, justifique detalhadamente o erro sintático:

Instrução	Status (C/I)	Justificativa Teórica
a) int a;	[Correto / Incorreto]	[Preencher]
b) float b;	[Correto / Incorreto]	[Preencher]
c) double float c;	[Correto / Incorreto]	[Preencher]
d) unsigned char d;	[Correto / Incorreto]	[Preencher]
e) unsigned e;	[Correto / Incorreto]	[Preencher]
f) long float f;	[Correto / Incorreto]	[Preencher]
g) long g;	[Correto / Incorreto]	[Preencher]
h) long double h;	[Correto / Incorreto]	[Preencher]

Questão 13. No desenvolvimento de programas em C, o que são conceitualmente os arquivos de inclusão (headers com extensão .h)?

- a) São bibliotecas pré-compiladas em formato binário contendo funções estruturadas.
- b) São utilitários do sistema que realizam a linkedição dos programas.
- c) São arquivos de texto ASCII padrão contendo protótipos de funções, definições de constantes, macros e tipos.
- d) São módulos de controle executados diretamente pelo microprocessador em tempo de execução.

Questão 14. Qual é o papel e o objetivo principal do programador ao incluir arquivos de cabeçalho (como <stdio.h>)?

- a) Instruir o compilador a carregar as definições das funções da biblioteca padrão antes de compilar o código-fonte.
- b) Linkeditar os arquivos binários do projeto automaticamente.
- c) Executar e testar as saídas de vídeo diretamente no console de depuração.
- d) Converter automaticamente o código-fonte C em arquivos executáveis (.EXE).

Questão 15. A diretiva `#include`, amplamente utilizada no topo dos arquivos C, é classificada como:

- a) Uma instrução C nativa (compilada diretamente em linguagem de máquina).
- b) Uma instrução específica de linguagens de programação orientadas a objetos.
- c) Uma diretiva especial para o pré-processador C, executada antes da compilação.
- d) Um objeto de classe de armazenamento dinâmico na memória heap.

Questão 16. As diretivas de pré-processador em C (todas iniciadas com o caractere `#`) são lidas e interpretadas pelo:

- a) Linkeditor do sistema no momento de montagem do arquivo executável final.
- b) Microprocessador diretamente em tempo de execução.
- c) Pré-processador (fase do compilador que altera o programa-fonte antes da compilação propriamente dita).
- d) Depurador integrado da IDE durante os testes de execução.

Questão 17. Dentre as instruções de escrita abaixo, quais estão sintaticamente corretas? O que essas variações demonstram sobre a flexibilidade de espaçamento e formatação do compilador C?

- a) `printf ( "Primeiro programa" );`
- b) `printf( "Primeiro programa" );`
- c) `printf("Primeiro programa");`
- d) `printf "Primeiro programa" ;`

Questão 18. Desenvolva um programa completo em C que declare variáveis de ponto flutuante para os seguintes itens e seus preços unitários: Lápis (4.88), Borrachas (234.54), Canetas (42.04), Cadernos (8.00) e Fitas (13.05). Utilize a função `printf()` para exibir esses dados no console em formato de tabela, alinhados à direita, com largura mínima de campo de 12 caracteres e precisão de duas casas decimais, conforme a saída abaixo:

Lapis	4.88
Borrachas	234.54
Canetas	42.04
Cadernos	8.00
Fitas	13.05

Questão 19. Escreva um programa em C que contenha uma única chamada de função para impressão (um único `printf()`) e produza exatamente a saída formatada abaixo com tabulação em cascata:

```
um
dois
três
```

Questão 20. Caracteres gráficos baseados na tabela ASCII estendida (Codepage 437) podem ser usados para desenhar molduras e caixas de diálogo na tela de modo console. Desenvolva um programa em C que produza uma moldura simples com exatamente 4 caracteres de largura por 4 de altura. Dica:

utilize constantes de caracteres em hexadecimal para representar os cantos e as retas horizontais/verticais:

```
Cantos Superiores: Esquerdo = \xC9, Direito = \xBB
Cantos Inferiores: Esquerdo = \xC8, Direito = \xBC
Linha Horizontal: \xCD, Linha Vertical: \xBA
```

Questão 21. Desenvolva três versões independentes de programas em C para produzir no console a saída de texto abaixo. A primeira versão deve usar uma única chamada de `printf()`; a segunda deve usar exatamente duas instruções de impressão independentes; e a terceira deve desenhar as frases emolduradas utilizando caracteres gráficos de caixa:

```
Treinamento em programação.
Linguagem C.
```

Questão 22. Desenhe no console um carro e uma caminhonete utilizando caracteres de bloco e de controle estudados no capítulo. Utilize sequências de escape em hexadecimal (como `\xDC` e `\xDF`) para renderizar a seguinte arte gráfica:



Questão 23. Escreva um programa em C que exiba no console uma figura simples (como uma caixa retangular vazia de 5x5) formada inteiramente por uma letra de sua escolha (como 'X'), similar ao modelo a seguir:

```
XXXXX
X  X
X  X
X  X
X  X
XXXXX
```

Questão 24. Desenvolva um programa em C que organize dados de notas escolares em uma tabela no console. Seu programa deve usar especificadores de formato e largura de campos para que as colunas fiquem perfeitamente alinhadas, gerando a saída mostrada abaixo:

```
ALUNO(A)    NOTA
=====
ALINE       9.0
```

MÁRIO	DEZ
SÉRGIO	4.5
SHIRLEY	7.0

Questão 25. Escreva um programa em C contendo uma única instrução printf() que desenhe a letra C em formato ampliado, utilizando a própria letra para a sua composição, conforme o modelo abaixo:

```
CCCCC
C
C
CCCCC
```

Questão 26. Elabore um programa em C que utilize caracteres para desenhar um 'pinheiro de Natal' estilizado no console. Enriqueça o desenho adicionando outros caracteres (como *, o, +) simulando enfeites espalhados pela árvore:

```
X
  X*X
  X+XoX
  X*X+X*X
XXXXXXXXX
  XX
  XX
  XXXX
```

Questão 27. Escreva um programa em C que solicite ao usuário (usando a função scanf()) um valor inteiro correspondente a um intervalo de tempo em segundos. O programa deve processar esse dado, calcular e exibir o equivalente formatado em Horas, Minutos e Segundos restantes (Exemplo: 3665 segundos correspondem a 1 hora, 1 minuto e 5 segundos).

Questão 28. Desenvolva um programa em C que leia três valores numéricos inteiros fornecidos pelo usuário através do teclado, calcule a média aritmética simples desses valores como um número real de dupla precisão (double) e exiba o resultado final na tela formatado com exatamente duas casas decimais.

Fim da Lista de Exercícios • Cesar School 2026.2
Desejamos excelente estudo e prática de programação!